

Project Report: Campaign Analytics Dashboard

1. Executive Summary

The Campaign Analytics Dashboard is a robust, full-stack application engineered to provide high-performance visualization and management of marketing campaign data. Built using a modern decoupled architecture, the platform empowers users with real-time filtering, multi-view data displays (Table and Card), and a scalable backend.

The project demonstrates a commitment to clean code, modular design, and industry-standard software engineering practices.

2. Project Overview

The primary goal of the Campaign Analytics Dashboard is to transform raw marketing data into actionable insights.

Key Functionalities

- **Real-time Filtering:** Dynamic updates based on campaign status (Active, Paused, etc.)
- **Dual Display Modes:**
 - Table View for detailed analysis
 - Card View for quick visual summaries
- **Unified API:** A central RESTful service providing consistent data access

- **Cross-Platform Accessibility:** Responsive design optimized for desktop and mobile

Technology Stack

- **Frontend:** Next.js 14
 - **Backend:** FastAPI
 - **Database:** PostgreSQL / SQLite
-

3. SDSE Concepts: Application and Rationale

The development of the Campaign Analytics Dashboard was guided by key Software Development and Systems Engineering (SDSE) principles.

A. Client-Server Architecture (Decoupling)

- **Where:** Separation of **frontend/** and **backend/** directories
 - **What:** Two independent services communicating via HTTP/REST
 - **Why:** Enables independent scaling and updates for frontend and backend
-

B. RESTful API Design

- **Where:** **backend/main.py**
- **What:** Use of HTTP methods (GET) and query parameters
- **Why:** Ensures a standardized, predictable interface and future scalability

C. Object-Relational Mapping (ORM)

- Where: `backend/models.py`, `backend/database.py` (SQLAlchemy)
- What: Database interaction using Python classes
- Why: Improves readability, reduces SQL complexity, and enhances security

D. Component-Based UI Architecture

- Where: `frontend/app/components/` (`CampaignTable.tsx`, `CampaignCard.tsx`)
- What: Modular UI components
- Why: Promotes DRY principle, improves reusability and maintainability

E. Software Design Modeling (UML Diagrams)

- Where: `Diagrams/` directory
- What: Sequence, Component, Class, and Use Case diagrams
- Why: Helps visualize system design and improves stakeholder communication

F. Data Seeding and Environment Consistency

- Where: `backend/populate_db.py`, `backend/setup.sql`
- What: Automated database initialization with mock data

- **Why:** Ensures consistent environments for development and testing
-

G. Security & Cross-Origin Resource Sharing (CORS)

- **Where:** Middleware in `backend/main.py`
 - **What:** Controlled access between frontend and backend
 - **Why:** Prevents unauthorized API access and enhances security
-

4. Technical Specifications

- **Frontend:** Next.js 14 (App Router), TypeScript, Tailwind CSS
 - **Backend:** FastAPI (Python), Uvicorn
 - **Database:** PostgreSQL (Production), SQLite (Development)
 - **ORM:** SQLAlchemy
 - **Deployment:**
 - **Frontend** → Vercel
 - **Backend** → Render
-

5. Conclusion

The Campaign Analytics Dashboard is a strong example of a modern, well-engineered web application. By applying SDSE principles such as RESTful design, ORM abstraction, and component-based architecture, the system achieves high scalability and maintainability.

The use of detailed UML diagrams further reinforces a structured engineering approach, ensuring clarity and readiness for future enhancements.

Report Prepared By:

Campaign Dashboard Engineering Team